Machine Learning Project 2026

Kaggle

1 Contexte et description du challenge

Ce projet s'inscrit dans le cadre d'une compétition Kaggle portant sur l'analyse de données issues d'imagerie satellite multi-dates. L'objectif est de classifier des zones géographiques en six catégories : *Demolition*, *Road*, *Residential*, *Commercial*, *Industrial* et *Mega Projects*.

Les données fournies sont issues de traitements de vision par ordinateur appliqués à des images satellites, et comprennent plusieurs types d'informations : des polygones géographiques irréguliers, des variables catégorielles décrivant l'évolution du statut de construction sur cinq dates, des caractéristiques urbaines et géographiques du voisinage, ainsi que des mesures spectrales (moyennes et écarts-types RGB) à différentes dates.

Le problème est un problème de classification supervisée multiclasse avec des données tabulaires de grande dimension et fortement déséquilibrées.

Les labels du jeu de test ne sont pas accessibles localement. Les performances sur ce jeu sont donc obtenues en soumettant les prédictions sur la plateforme Kaggle, qui calcule ensuite le score en comparant ces prédictions aux labels réels du test.

2 Métrique d'évaluation

La métrique officielle de la compétition est le **Mean F1-Score (macro-F1)**, qui correspond à la moyenne des F1-scores calculés indépendamment pour chaque classe.

Cette métrique est particulièrement adaptée aux données déséquilibrées, car elle donne le même poids à chaque classe, indépendamment de sa fréquence. Ainsi, les performances rapportées dans ce travail sont principalement évaluées en macro-F1.

Sur la plateforme Kaggle, le score correspond également à ce macro-F1. Le leaderboard public est calculé sur environ 30% des données de test, tandis que le leaderboard privé final est calculé sur les 70% restants. Cela peut entraîner des écarts entre les scores observés pendant la compétition et le score final.

3 Feature Engineering et Prétraitement des Données

Le jeu de données initial contenait 296 146 observations et 45 variables pour l'entraînement, et 120 526 observations et 44 variables pour le test. Afin d'améliorer la capacité prédictive du modèle, un travail approfondi de *feature engineering* a été réalisé à partir des dimensions géométriques, temporelles et spectrales.

Les géométries ont été reprojetées en EPSG :3857 afin d'obtenir des mesures fiables de surface et de périmètre. À partir de celles-ci, plusieurs indicateurs de forme ont été construits, notamment la compacité, la *solidity*, les ratios de boîte englobante, la rectangularité ainsi que des transformations logarithmiques. En parallèle, les dates ont été converties puis réordonnées chronologiquement pour garantir la cohérence entre statuts de construction et valeurs spectrales associées. Cette réorganisation a permis de calculer des variables dynamiques telles que la durée totale du projet, la progression des statuts, la vitesse de construction et différentes variations temporelles.

Les variables catégorielles multi-étiquettes ont été transformées via un encodage multi-hot, puis alignées entre les jeux d'entraînement et de test. À partir des mesures RGB disponibles sur cinq dates, des indicateurs temporels ont également été extraits : moyennes, pentes d'évolution, amplitudes,

différences successives et ratios spectraux. Enfin, plusieurs interactions entre géométrie, dynamique de statut et signaux spectraux ont été introduites afin de capturer des effets non linéaires complexes.

Après cette phase d'enrichissement, le nombre de variables est passé à 135 pour les deux jeux de données (296 146 $\times$ 135 pour l'entraînement et 120 526 $\times$ 135 pour le test). Une étape de sélection a ensuite été appliquée, consistant à supprimer les variables à faible variance ainsi que celles fortement corrélées ($\rho > 0.95$). Le jeu final utilisé pour l'apprentissage contient 92 variables, soit une dimension finale de 296 146 $\times$ 92 pour l'entraînement et 120 526 $\times$ 92 pour le test.

Ce pipeline permet d'obtenir un jeu de données enrichi, cohérent temporellement et optimisé pour l'apprentissage supervisé tout en maîtrisant la complexité dimensionnelle.

4 Choix du Modèle

Plusieurs familles de modèles ont été évaluées après la phase de *feature engineering*. Les SVM n'ont pas été retenus en raison de la très forte dimension des données, rendant leur coût computationnel peu adapté à ce contexte. Le modèle KNN a également été écarté car il nécessite le calcul d'un grand nombre de distances dans un espace de grande dimension, ce qui le rend peu scalable malgré une performance atteignant environ 0.81 en macro-F1 sur le leaderboard public Kaggle avec nos features.

Les réseaux de neurones ont été testés mais se sont révélés difficiles à exploiter efficacement. Un premier modèle simple a atteint une performance de 0.37 en macro-F1 sur le leaderboard public Kaggle, puis 0.52 après une complexification de l'architecture. Cependant, ces gains se sont accompagnés d'un temps d'entraînement élevé et d'une forte sensibilité aux hyperparamètres. Finalement, les modèles à base d'arbres boostés (LightGBM et XGBoost) ont été retenus, car ils offrent un excellent compromis entre performance, robustesse et efficacité sur des données tabulaires de grande dimension, tout en capturant efficacement des relations non linéaires complexes.

5 Premier Modèle de Boosting : LightGBM

Le premier modèle de classification retenu a été un *Gradient Boosting* basé sur LightGBM, entraîné sur les 92 features finales avec une validation croisée stratifiée à 5 folds. Les paramètres ont été ajustés pour gérer le déséquilibre des classes et limiter l'overfitting, notamment via `class_weight=balanced`, `feature_fraction=0.8` et `subsample=0.8`. Chaque fold a été entraîné avec early stopping, et les prédictions finales sur le test set ont été obtenues en moyennant les probabilités de tous les folds.

Le modèle a atteint une accuracy globale de 0.773 et un F1-score macro de 0.536 en validation croisée. La distribution des classes dans l'ensemble d'entraînement est fortement déséquilibrée : les classes *Residential* et *Commercial* représentent 50% et 34% des échantillons, tandis que les classes *Industrial* et *Mega Projects* ne représentent que 0.4% et 0.05%. Malgré ce déséquilibre, le modèle a montré une bonne capacité à capturer les classes majoritaires.

Sur Kaggle, il obtient un score de 0.883 sur le leaderboard public, ce qui indique une bonne capacité de généralisation, bien que ce score soit calculé sur un sous-ensemble du jeu de test.

L'approche LightGBM permet ainsi de traiter efficacement un grand nombre de features et d'observations, de gérer le déséquilibre et de modéliser des interactions non linéaires complexes tout en restant relativement rapide à entraîner par rapport à des réseaux de neurones.

Une recherche d'hyperparamètres a également été effectuée avec Optuna afin d'optimiser le LightGBM. Malgré une exploration de différents taux d'apprentissage, nombres de feuilles et fractions de features, les améliorations observées étaient très limitées (changements du troisième chiffre après la virgule) et provoquaient un léger surapprentissage, notamment sur les classes minoritaires. Cette observation a conduit à privilégier XGBoost, une autre implémentation de boosting plus robuste, offrant un meilleur compromis entre performance globale et gestion du déséquilibre des classes.

6 Modèle XGBoost avec pondération et optimisation conjointe des seuils

Nous utilisons un modèle **XGBoost** pour une classification multiclasse à 6 classes. L'objectif `multi:softprob` produit, pour chaque observation x, un vecteur de probabilités

$$\mathbf{p}(x) = \big(P(y=0|x), \ldots, P(y=5|x)\big).$$

Pondération pour données déséquilibrées. Les classes 4 et 5 sont peu représentées dans l'ensemble d'entraînement. Afin de corriger le biais vers les classes majoritaires, chaque observation est pondérée par l'inverse de la fréquence de sa classe :

$$w_c = \min\left(\frac{n}{k\,n_c},\, w_{\max}\right),$$

où n_c est le nombre d'exemples de la classe c, n le nombre total d'observations et k le nombre de classes. Un plafond $w_{\max}$ évite des poids extrêmes.

D'autres approches pour gérer le déséquilibre ont été explorées, notamment le sur-échantillonnage (SMOTE) et le re-sampling. Cependant, ces méthodes n'ont pas permis d'améliorer significativement le macro-F1, et ont parfois dégradé les performances, ce qui nous a conduit à privilégier une approche par pondération.

Cette pondération améliore la sensibilité du modèle aux classes rares. Cependant, en phase de test, elle induit une **sur-prédiction** des classes 4 et 5. En effet, leur importance accrue dans la fonction de perte pousse le modèle à leur attribuer des probabilités élevées, même lorsque leur fréquence réelle est faible.

Règle de décision avec seuils pour les classes rares. Afin de contrôler cette sur-prédiction, l'attribution des classes 4 et 5 est soumise à des seuils t_4 et t_5. Ces classes ne peuvent être prédites que si leurs probabilités dépassent ces seuils :

$$P(y=4|x) > t_4, \qquad P(y=5|x) > t_5.$$

Sinon, la prédiction est restreinte aux classes $\{0,1,2,3\}$. Si les deux seuils sont franchis, la classe la plus probable entre 4 et 5 est choisie.

Optimisation conjointe du modèle et des seuils. Les hyperparamètres du modèle sont optimisés avec **Optuna** en validation croisée stratifiée (3 folds). Pour chaque essai, les probabilités *out-of-fold* sont calculées, puis les seuils (t_4, t_5) sont recherchés par grille afin de maximiser le **macro-F1**. La métrique optimisée par Optuna est donc le macro-F1 *après application de la règle de décision*, ce qui permet d'ajuster simultanément :

Résultats et paramètres optimaux. L'optimisation conjointe du modèle et des seuils via Optuna a conduit à un **macro-F1 de validation de 0.574** en validation croisée stratifiée.

Les hyperparamètres optimaux du modèle XGBoost sont :

- `learning_rate` = 0.0167
- `max_depth` = 9
- `min_child_weight` = 4.01
- `subsample` = 0.6498
- `colsample_bytree` = 0.6242
- `gamma` = 0.3709
- `alpha` = 6.1107
- `lambda` = 0.0159
- `max_delta_step` = 3.086
- plafond de pondération $w_{\max} = 39.79$

Le nombre moyen d'itérations optimales avec early stopping est d'environ **4997** arbres.
Les seuils optimaux appliqués aux classes rares sont :

$$t_4 = 0.225, \qquad t_5 = 0.10.$$

Ces paramètres permettent d'obtenir un compromis robuste entre performance globale et détection des classes minoritaires.

Modèle final et prédiction. Après sélection des meilleurs paramètres, le modèle est ré-entraîné sur l'ensemble des données avec les poids optimaux. Les seuils optimisés sont ensuite appliqués aux probabilités prédites pour produire les classes finales. Cette approche permet de conserver le bénéfice de la pondération pour les classes rares tout en limitant leur sur-prédiction. Le modèle final atteint un score de 0.91 sur le leaderboard public Kaggle. Après la fin de la compétition, le score sur le leaderboard privé (70% des données) est de **0.84**, reflétant plus fidèlement la performance réelle du modèle.

7 Conclusion

Le travail de prétraitement et d'enrichissement des données a permis de construire un jeu de variables cohérent, riche et adapté aux modèles supervisés. Les modèles de boosting, LightGBM puis XGBoost, se sont révélés les plus performants pour ce jeu de données tabulaire et déséquilibré. Le macro-F1 étant la métrique principale, nous avons privilégié des modèles performants sur les classes minoritaires, même au détriment de l'accuracy globale. L'optimisation conjointe des hyperparamètres et des seuils pour les classes rares a permis d'améliorer le macro-F1 et de produire des prédictions robustes tout en maîtrisant la sur-prédiction des classes minoritaires. Cette approche illustre l'importance d'un pipeline complet combinant *feature engineering*, modélisation et post-traitement pour maximiser la performance sur des problèmes réels de classification multiclasse.